

第3章 选择与循环

《Python程序设计，董付国》

3.1 条件表达式

❖ 几乎所有的Python合法表达式都可以作为条件表达式.

- 关系运算符: `>`、`<`、`==`、`<=`、`>=`、`!=`，可以连续使用，如

```
>>> 1 < 2 < 3          # 等价于 1<2 and 2<3
```

```
True
```

```
>>> 1 < 2 > 3
```

```
False
```

```
>>> 1 < 3 > 2
```

```
True
```

- 测试运算符: `in`、`not in`、`is`、`is not`，优先级与关系运算符一样
- 逻辑运算符: `and`、`or`、`not`，注意短路求值

3.1 条件表达式

- 在选择和循环结构中，条件表达式的值**只要不是**False、0（或0.0、0j等）、空值None、空列表、空元组、空集合、空字典、空字符串、空range对象或其他空容器对象，Python解释器均认为与True等价。map、zip、filter、enumerate对象、生成器对象等迭代器对象都等价于True。
- 所有能够使得内置函数bool()返回True的对象作为条件表达式时都等价于True。

3.1 条件表达式

```
>>> if 3:                                # 使用整数作为条件表达式, 非0等价于True
    print(5)

5
>>> a = [1, 2, 3]
>>> if a:                                # 使用列表作为条件表达式, 非空列表等价于True
    print(a)

[1, 2, 3]
>>> a = []
>>> if a:                                # 空列表等价于False
    print(a)
else:
    print('empty')
```

empty

3.1 条件表达式

```
>>> i = s = 0
>>> while i <= 10:           # 使用关系表达式作为条件表达式
    s = s + i
    i = i + 1
>>> print(s)
```

55

```
>>> i = s = 0
>>> while True:             # 使用常量True作为条件表达式
    s = s + i
    i = i + 1
    if i > 10:
        break
>>> print(s)
```

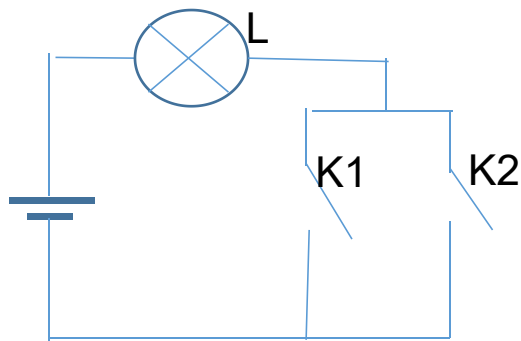
55

```
>>> s = 0
>>> for i in range(0, 11, 1): # 遍历可迭代对象中的所有元素
    s = s + i
>>> print(s)
```

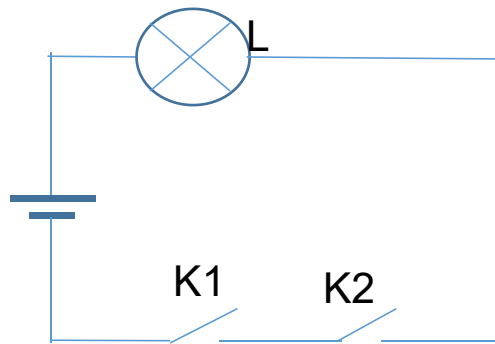
55

3.1 条件表达式

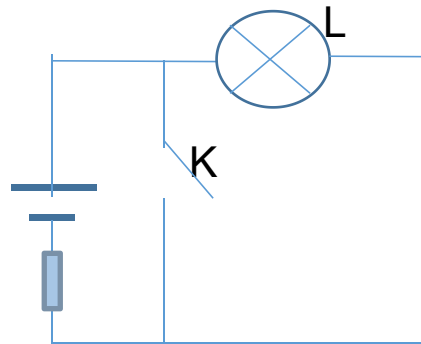
- 逻辑运算符and和or以及关系运算符具有惰性求值特点，只计算必须计算的表达式。



(1) or, 并联电路



(2) and, 串联电路



(3) not, 短路

3.1 条件表达式

- 以“and”为例，对于表达式“表达式1 and 表达式2”而言，如果“表达式1”的值为“False”或其他等价值时，不论“表达式2”的值是什么，整个表达式的值都是“False”，此时“表达式2”的值无论是什么都不影响整个表达式的值，因此将不会被计算，从而减少不必要的计算和判断。
- 在设计条件表达式时，如果能够大概预测不同条件失败的概率，并将多个条件根据“and”和“or”运算的短路求值特性来组织先后顺序，可以大幅度提高程序运行效率。

3.1 条件表达式

- 在Python中，条件表达式中不允许使用赋值分隔符“=”，可以使用赋值运算符。

```
>>> if a=3:
```

```
SyntaxError: invalid syntax
```

```
>>> if (a=3) and (b=4):
```

```
SyntaxError: invalid syntax
```

```
>>> if a:=3:
```

```
    print('ok')
```

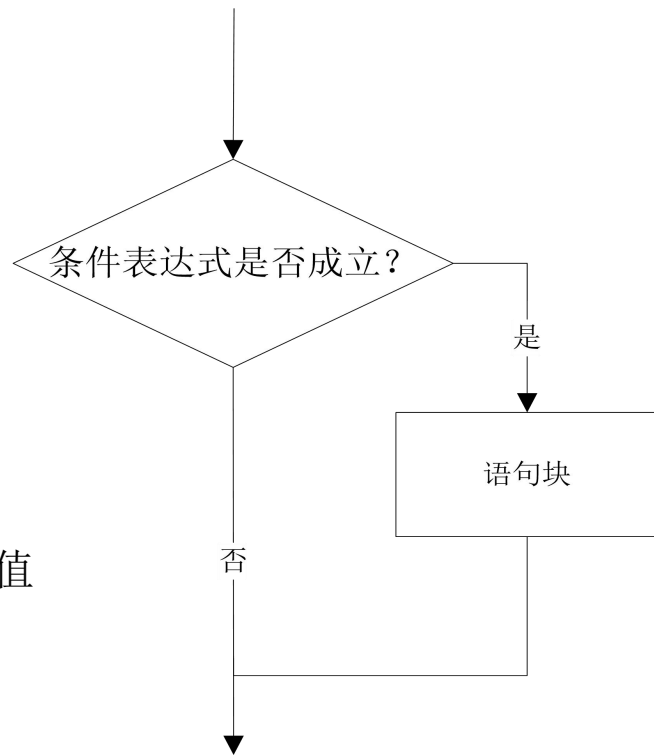
输出ok，同时创建变量a

ok

3.2.1 单分支选择结构

if 表达式:
 语句块

```
x = input('Input two integers:')  
a, b = map(int, x.split())  
if a > b:  
    a, b = b, a      # 序列解包, 交换两个变量的值  
print(a, b)
```

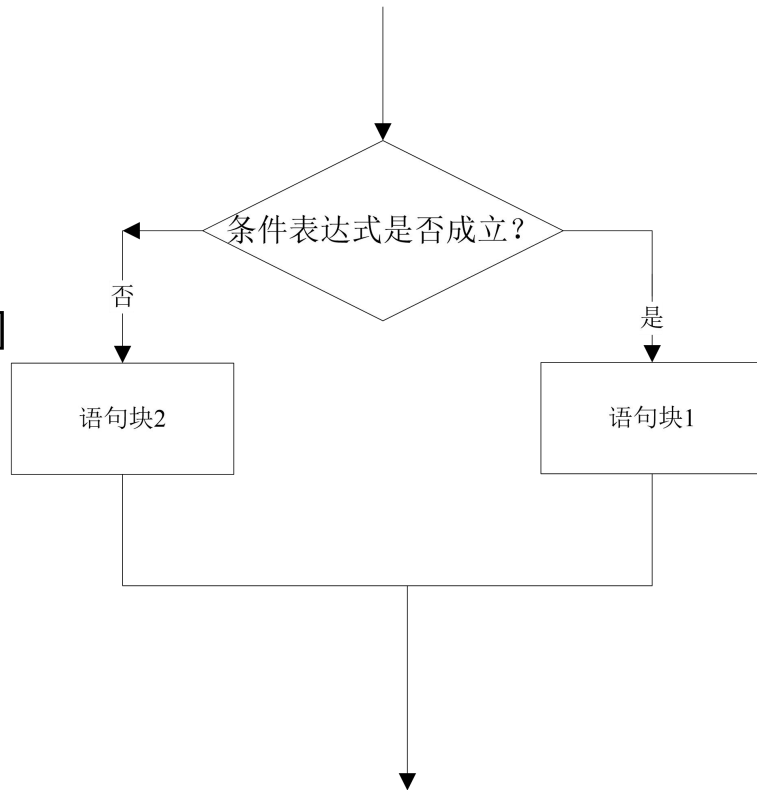


3.2.2 双分支结构

```
if 表达式:  
    语句块1  
else:  
    语句块2
```

```
>>> chTest = ['1', '2', '3', '4', '5']  
>>> if chTest:  
    print(chTest)  
else:  
    print('Empty')
```

```
['1', '2', '3', '4', '5']
```



3.2.2 双分支结构

■Python还支持如下形式的表达式：

```
value1 if condition else value2
```

当条件表达式condition的值与True等价时，表达式的值为value1，否则表达式的值为value2。在value1和value2中还可以使用复杂表达式，包括函数调用和基本输出语句。这个结构的表达式也具有惰性求值的特点。

```
>>> a = 5
>>> print(6 if a>3 else print(5))
6
>>> print(6 if a>3 else 5)
6
>>> b = 6 if a>13 else 9
>>> b
9
```

3.2.2 双分支结构

此时还没有导入math模块

```
>>> x = math.sqrt(9) if 5>3 else random.randint(1, 100)
```

NameError: name 'math' is not defined

```
>>> import math
```

此时还没有导入random模块，但由于条件表达式5>3的值为True，所以可以正常运行

```
>>> x = math.sqrt(9) if 5>3 else random.randint(1, 100)
```

此时还没有导入random模块，由于条件表达式2>3的值为False，需要计算第二个表达式的值，因此出错

```
>>> x = math.sqrt(9) if 2>3 else random.randint(1, 100)
```

NameError: name 'random' is not defined

```
>>> import random
```

```
>>> x = math.sqrt(9) if 2>3 else random.randint(1, 100)
```

3.2.3 嵌套的分支结构

```
if 表达式1:  
    语句块1  
elif 表达式2:  
    语句块2  
elif 表达式3:  
    语句块3  
else:  
    语句块4
```

其中，关键字**elif**是else if的缩写。

3.2.3 嵌套的分支结构

- 将成绩从百分制变换到等级制。

```
def func(score):  
    if score > 100:  
        return 'wrong score.must <= 100.'  
    elif score >= 90:  
        return 'A'  
    elif score >= 80:  
        return 'B'  
    elif score >= 70:  
        return 'C'  
    elif score >= 60:  
        return 'D'  
    elif score >= 0:  
        return 'E'  
    else:  
        return 'wrong score.must >0'
```

3.2.3 嵌套的分支结构

```
if 表达式1:  
    语句块1  
    if 表达式2:  
        语句块2  
    else:  
        语句块3  
else:  
    if 表达式4:  
        语句块4
```

The diagram illustrates the nested branching structure using vertical lines and numbers to represent indentation levels. A vertical line on the left is labeled '1'. To its right, another vertical line is labeled '2'. To the right of the '2' line, a third vertical line is labeled '3'. The code is aligned with these lines: 'if 表达式 1:' is at level 1, '语句块 1' is at level 1, 'if 表达式 2:' is at level 2, '语句块 2' is at level 2, 'else:' is at level 2, '语句块 3' is at level 2, 'else:' is at level 1, 'if 表达式 4:' is at level 2, and '语句块 4' is at level 2.

```
if 表达式 1:  
    语句块 1  
    if 表达式 2:  
        语句块 2  
    else:  
        语句块 3  
else:  
    if 表达式 4:  
        语句块 4
```

注意：缩进必须要正确并且一致。

3.2.3 嵌套的分支结构

- 使用嵌套的选择结构实现百分制成绩到等级制的转换。

```
def func(score):  
    degree = 'DCBAAE'  
    if score > 100 or score < 0:  
        return 'wrong score.must between 0 and 100.'  
    else:  
        index = (score - 60) // 10  
        if index >= 0:  
            return degree[index]  
        else:  
            return degree[-1]
```


3.2.4 多分支选择结构

- 在Python 3.9以及之前的版本中，没有提供真正意义上的多分支选择结构，如果确实需要的话，可以通过字典构造跳转表来实现。例如下面的代码，

```
status = {200:'ok', 201:'Created', 202:'Accepted',  
          203:'Non-Authoritative Information', 204:'No Content'}  
s = int(input('请输入状态码: '))  
print('对应的状态为: ', status.get(s, 'unknown'))  
funcs = {'1': lambda x: x**1, '2': lambda x: x**2,  
          '3': lambda x: x**3, '4': lambda x: x**4}  
k = input('请输入一个数字: ')  
func = funcs.get(k)  
if func:  
    print(func(int(k)))  
else:  
    print('输入错误。')
```

3.2.4 多分支选择结构

- Python 3.10新增了软关键字（只在特定场合作为关键字，普通场合也可以作变量名）**match**和**case**，实现了真正意义上的多分支选择结构。

```
code = int(input('请输入HTTP状态码: '))
match code:
    case 200:
        print('ok')
    case 201:
        print('Created')
    case 202:
        print('Accepted')
    case 203:
        print('Non-Authoritative Information')
    case 204:
        print('No Content')
    case 401 | 403 | 404:
        # 同时匹配401、403、404这三种情况
        print('Not allowed')
    case _:
        # 通配符，表示任意内容，如果前面的都不匹配，就执行这里的代码
        print('Sorry, please try later.')
```

3.2.4 多分支选择结构

- 下面的代码演示了下画线在元组中表示任意内容的用法：

```
while (point:=input('表示三维空间坐标的元组，0表示结束： ')) != '0':  
    point = eval(point)  
    match point:  
        case (0, 0, 0):  
            print('坐标原点')  
        case (0, _, _):  
            print('YOZ平面上的点')  
        case (_, 0, _):  
            print('XOZ平面上的点')  
        case (_, _, 0):  
            print('XOY平面上的点')
```

3.2.4 多分支选择结构

- 在`match...case...`多分支选择结构中，下画线除了上面的用法之外，还可以和星号组合使用，例如下面的代码中“* ”表示从当前位置往后还有0到任意多项：

```
while (content:=eval(input('请输入列表: '))) != 0:
```

```
    match content:
```

```
        case [1, 2, 3, 4, *  ]:
```

```
            print('前4项匹配成功')
```

```
        case [1, 2, 3, *  ]:
```

```
            print('前3项匹配成功')
```

```
        case [1, 2, *  ]:
```

```
            print('前2项匹配成功')
```

```
        case [1, *  ]:
```

```
            print('前1项匹配成功')
```

```
        case [*  ]:
```

```
            print('匹配失败')
```

```
        case   :
```

```
            print('格式不对')
```

3.2.4 多分支选择结构

- 在下面的代码中，使用if对当前匹配项进行约束，如果条件不成立就继续检查下一项是否匹配，其中的x和y可以是任意变量名。

```
match (3, 5):  
    case (x, y) if x<y:  
        print('<')  
    case (x, y) if x==y:  
        print('==')  
    case (x, y) if x>y:  
        print('>')
```

3.2.5 选择结构应用

- 例3-1 面试资格确认。

```
age, subject, college = 24, '计算机', '非重点'
if (age > 25 and subject=='电子信息工程') or\
    (college=='重点' and subject=='电子信息工程') or\
    (age<=28 and subject=='计算机'):
    print('恭喜, 你已获得我公司的面试机会!')
else:
    print('抱歉, 你未达到面试要求')
```

3.2.5 选择结构应用

- **例3-2** 用户输入若干个分数，每输入一个分数后询问是否继续输入下一个分数，回答“yes”就继续输入下一个分数，回答“no”就停止输入分数，计算所有分数的平均分。

3.2.5 选择结构应用

```
numbers = []                                # 使用列表存放临时数据
while True:
    x = input('请输入一个成绩: ')
    try:                                    # 异常处理结构
        numbers.append(float(x))
    except:
        print('不是合法成绩')
    while True:
        flag = input('继续输入吗? (yes/no) ').lower()
        if flag not in ('yes', 'no'):      # 限定用户输入内容必须为yes或no
            print('只能输入yes或no')
        else:
            break
    if flag == 'no':
        break

print(sum(numbers)/len(numbers))
```


3.2.5 选择结构应用

- 例3-3 编写程序，判断今天是今年的第几天。

```
import time

def demo(year, month, day):
    day_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
    if year%400==0 or (year%4==0 and year%100!=0): # 判断是否为闰年
        day_month[1] = 29
    if month == 1:
        return day
    else:
        return sum(day_month[:month-1]) + day

year, month, day = time.localtime()[0:3]
print(demo(year, month, day))
```

3.2.5 选择结构应用

- 其中闰年判断可以直接使用calendar模块的方法。

```
>>> import calendar
>>> calendar.isleap(2016)
True
>>> calendar.isleap(2023)
False
```

3.2.5 选择结构应用

- 使用datetime模块直接查看

```
>>> import datetime
```

```
>>> datetime.date.today().timetuple().tm_yday  
# 查看今天是今年的第几天
```

309

```
>>> datetime.date(2023,3,5).timetuple().tm_yday  
# 查看指定日期是所在年份的第几天
```

64

3.2.5 选择结构应用

- 也可以使用datetime模块提供的相关功能来计算

```
>>> today = datetime.date.today()
>>> today
datetime.date(2019, 9, 8)
>>> firstDay = datetime.date(today.year, 1, 1)
>>> firstDay
datetime.date(2019, 1, 1)
>>> daysDelta = today-firstDay + datetime.timedelta(days=1)
>>> daysDelta.days
251
```

3.2.5 选择结构应用

- `datetime`还提供了其他功能

```
>>> now = datetime.datetime.now()
```

```
>>> now
```

```
datetime.datetime(2023, 9, 8, 22, 50, 32, 872739)
```

```
>>> now.replace(second=30) # 替换日期时间中的秒
```

```
datetime.datetime(2023, 9, 8, 22, 50, 30, 872739)
```

```
>>> now + datetime.timedelta(days=5) # 计算5天后的日期时间
```

```
datetime.datetime(2023, 9, 13, 22, 50, 32, 872739)
```

```
>>> now + datetime.timedelta(weeks=-5) # 计算5周前的日期时间
```

```
datetime.datetime(2023, 8, 4, 22, 50, 32, 872739)
```

3.2.5 选择结构应用

```
>>> now + datetime.timedelta(days=-2, seconds=35) # 差35秒两天前
datetime.datetime(2023, 9, 6, 22, 51, 7, 872739)
>>> now + datetime.timedelta(weeks=-2, seconds=35) # 差35秒两周前
datetime.datetime(2023, 8, 25, 22, 51, 7, 872739)
```

3.2.5 选择结构应用

- 补充：计算两个日期之间相差多少天。

```
from datetime import date
```

```
def daysBetween(year1, month1, day1, year2, month2, day2):  
    dif = date(year1, month1, day1)  
    dif = dif - date(year2, month2, day2)  
    return abs(dif.days)
```

```
print(daysBetween(2022, 12, 11, 2022, 11, 27))  
print(daysBetween(2025, 11, 11, 2016, 12, 6))
```

3.2.5 选择结构应用

- 补充：计算某年第n个周五是几月几号。

```
from datetime import date, timedelta

def getDate(year, weeks, weekday):
    start = date(year, 1, 1)
    # 查找第一个周weekday是1月几日
    for i in range(7):
        if start.isoweekday() == weekday:
            break
        start = start + timedelta(days=1)
    return start + timedelta(weeks=weeks-1)

print(getDate(2023, 22, 5))
```


3.3.1 for循环与while循环

- Python提供了两种基本的循环结构语句——while和for。
- while循环一般用于循环次数难以提前确定的情况，也可以用于循环次数确定的情况。
- for循环一般用于循环次数可以提前确定的情况，尤其是用于枚举序列或可迭代对象中的元素。
- 一般优先考虑使用for循环。
- 相同或不同的循环结构之间都可以互相嵌套，实现更为复杂的逻辑。
- for循环和while循环都可以带else，也可以不带，取决于要解决的具体问题。

3.3.1 for循环与while循环

`while` 条件表达式:

 循环体

[`else`:

`else`子句代码块]

如果循环是因为`break`结束的，就不执行`else`中的代码

`for` 循环变量 `in` 可迭代对象:

 循环体

[`else`:

`else`子句代码块]

3.4 break和continue语句

- break语句在while循环和for循环中都可以使用，一般放在if选择结构中，一旦break语句被执行，将使得整个循环提前结束。
- continue语句的作用是终止当前循环，并忽略continue之后的语句，然后回到循环的顶端，提前进入下一次循环。
- 除非break语句让代码更简单或更清晰，否则不要轻易使用。

3.4 break和continue语句

■ 下面的代码用来计算小于100的最大素数，注意break语句和else子句的用法。

```
>>> for n in range(99, 1, -1):  
    for i in range(2, n):  
        if n%i == 0:  
            break  
    else:  
        print(n)  
        break
```

3.4 break和continue语句

- 删除上面代码中最后一个break语句，则可以用来输出100以内的所有素数。

```
>>> for n in range(100, 1, -1):  
    for i in range(2, n):  
        if n%i == 0:  
            break  
    else:  
        print(n, end=' ')
```

97 89 83 79 73 71 67 61 59 53 47 43 41 37 31 29 23 19 17 13 11 7 5 3 2

3.5 案例精选

- 例3-4 计算 $1+2+3+\cdots+100$ 的值。

```
s = 0
for i in range(1, 101):
    s = s + i
print('1+2+3+...+100 = ', s)
print('1+2+3+...+100 = ', sum(range(1,101)))
```

3.5 案例精选

- 例3-5 输出序列中的元素。

```
a_list = ['a', 'b', 'mpilgrim', 'z', 'example']  
for i, v in enumerate(a_list):  
    print('列表的第', i+1, '个元素是: ', v)
```

```
a_list = ['a', 'b', 'mpilgrim', 'z', 'example']  
for i, v in enumerate(a_list, start=1):  
    print(f'列表的第{i}个元素是: {v}')
```

3.5 案例精选

- 例3-6 求1~100之间能被7整除，但不能同时被5整除的所有整数。

```
for i in range(1, 101):  
    if i%7 == 0 and i%5 != 0:  
        print(i)
```

```
for num in range(7, 101, 7):  
    if num%5 != 0:  
        print(num)
```


3.5 案例精选

- **例3-7** 输出所有3位“水仙花数”。所谓n位水仙花数是指1个n位的十进制数，其各位数字的n次方之和等于该数本身。例如：153是水仙花数，因为 $153 = 1^3 + 5^3 + 3^3$ 。

3.5 案例精选

✓ 方法一:

```
for i in range(100, 1000):  
    bai, shi, ge = map(int, str(i))  
    if ge**3 + shi**3 + bai**3 == i:  
        print(i)
```

3.5 案例精选

✓ 方法二:

```
for num in range(100, 1000):  
    if sum(map(lambda x:int(x)**3, str(num))) == num:  
        print(num)
```

3.5 案例精选

✓ 扩展到n位水仙花数:

```
n = int(input('请输入位数: '))
for num in range(10**(n-1), 10**n):
    if sum(map(lambda i: int(i)**n, str(num))) == num:
        print(num)
```

3.5 案例精选

✓ 函数式编程:

```
n = int(input('请输入位数: '))
result = filter(lambda num: sum(map(lambda i: int(i)**n,
                                     str(num)))==num,
                range(10**(n-1), 10**n))
for num in result:
    print(num)
```

3.5 案例精选

- 例3-8 统计考试成绩中优、良、中、及格、不及格的人数。

✓ 方法一:

```
scores = [89, 70, 49, 87, 92, 84, 73, 71, 78, 81, 90, 37,  
          77, 82, 81, 79, 80, 82, 75, 90, 54, 80, 70, 68, 61]  
groups = {'优秀': 0, '良': 0, '中': 0, '及格': 0, '不及格': 0}  
for score in scores:  
    if score >= 90:  
        groups['优秀'] = groups['优秀'] + 1  
    elif score >= 80:  
        groups['良'] = groups['良'] + 1  
    elif score >= 70:  
        groups['中'] = groups['中'] + 1  
    elif score >= 60:  
        groups['及格'] = groups['及格'] + 1  
    else:  
        groups['不及格'] = groups['不及格'] + 1  
print(groups)
```

3.5 案例精选

✓ 方法二:

```
from itertools import groupby
```

```
scores = [89, 70, 49, 87, 92, 84, 73, 71, 78, 81, 90, 37,  
          77, 82, 81, 79, 80, 82, 75, 90, 54, 80, 70, 68, 61]
```

```
def classify(score):  
    if score >= 90: return '优秀'  
    elif score >= 80: return '良'  
    elif score >= 70: return '中'  
    elif score >= 60: return '及格'  
    else: return '不及格'
```

```
groups = {category:len(tuple(score))  
          for category, score in groupby(sorted(scores), classify)}  
print(groups)
```

3.5 案例精选

✓ 方法三:

```
from collections import Counter
from pandas import cut          # 需要先安装扩展库pandas才能使用

scores = [89, 70, 49, 87, 92, 84, 73, 71, 78, 81, 90, 37,
          77, 82, 81, 79, 80, 82, 75, 90, 54, 80, 70, 68, 61]
# 可以使用pandas中的函数value_counts()代替Counter类
groups = Counter(cut(scores, [0,60,70,80,90,101],
                    labels=['不及格','及格','中','良','优秀'],
                    right=False))

print(groups)
```


3.5 案例精选

- 例3-9 打印九九乘法表。

```
for i in range(1, 10):  
    for j in range(1, i+1):  
        print('{0}*{1}={2}'.format(i,j,i*j).ljust(6), end=' ')  
    print()
```

```
1*1=1  
2*1=2  2*2=4  
3*1=3  3*2=6  3*3=9  
4*1=4  4*2=8  4*3=12  4*4=16  
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25  
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36  
7*1=7  7*2=14  7*3=21  7*4=28  7*5=35  7*6=42  7*7=49  
8*1=8  8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64  
9*1=9  9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81
```

3.5 案例精选

- 例3-10 求200以内能被17整除的最大正整数。

```
for i in range(200, 0, -1):  
    if i%17 == 0:  
        print(i)  
        break
```

3.5 案例精选

- 例3-11 判断一个数是否为素数。

```
import math
```

```
n = int(input('Input an integer:'))
```

```
m = math.ceil(math.sqrt(n)+1)
```

```
for i in range(2, m):
```

```
    if n%i == 0 and i<n:
```

```
        print('No')
```

```
        break
```

```
else:
```

```
    print('Yes')
```

3.5 案例精选

- 例3-12 鸡兔同笼问题。假设共有鸡、兔30只，脚90只，求鸡、兔各有多少只。

```
for ji in range(0, 31):  
    if 2*ji + (30-ji)*4 == 90:  
        print('ji:', ji, ' tu:', 30-ji)  
        break
```

3.5 案例精选

- 补充：另类解法。

```
def demo(tui, jitu):  
    tu = (tui - jitu*2) / 2  
    if int(tu)==tu and 0<=tu<=jitu:  
        return (int(tu), int(jitu-tu))  
    else:  
        return 'Data Error'
```

$$\begin{cases} x + y = 30 \\ 2x + 4y = 90 \end{cases}$$

3.5 案例精选

- 例3-13 编写程序，输出由1、2、3、4这四个数字组成的每位数都不相同的的所有三位数。

```
digits = (1, 2, 3, 4)
for i in digits:
    for j in digits:
        for k in digits:
            if i!=j and j!=k and i!=k:
                print(i*100+j*10+k)
```

3.5 案例精选

- 从代码优化的角度来讲，上面这段代码并不是很好，其中有些判断完全可以在外层循环来做，尽量减少内循环中不必要的计算，从而提高运行效率。

```
digits = (1, 2, 3, 4)
for i in digits:
    for j in digits:
        if j == i:
            continue
        for k in digits:
            if k==i or k==j:
                continue
            print(i*100+j*10+k)
```

3.5 案例精选

- 当然，还可以进一步优化。

```
digits = (1, 2, 3, 4)
for i in digits:
    ii = i * 100
    for j in digits:
        if j == i:
            continue
        jj = ii + j * 10
        for k in digits:
            if k!=i and k!=j:
                print(jj + k)
```


3.5 案例精选

- 也可以使用集合实现同样功能。

```
digits = {1, 2, 3, 4}
for i in digits:
    ii = i * 100
    for j in digits-{i}:
        jj = ii + j * 10
        for k in digits-{i,j}:
            print(jj + k)
```

3.5 案例精选

- 还可以使用排列函数实现。

```
from itertools import permutations
```

```
digits = {1, 2, 3, 4}
```

```
for num in permutations(digits, 3):  
    num = int(''.join(map(str, num)))  
    print(num)
```

3.5 案例精选

- **例3-14** 有一箱苹果，4个4个地数最后余下1个，5个5个地数最后余下2个，9个9个地数最后余下7个。编写程序计算这箱苹果至少有多少个。
- 先确定除以9余7的最小整数，对这个数字重复加9，如果得到的数字除以5余2就停止；然后对得到的数字重复加45（5和9的最小公倍数），如果得到的数字除以4余1就停止。这时得到的数字就是题目的答案。

3.5 案例精选

```
from itertools import count

for num in count(16, 9):
    if num%5 == 2:
        break
for result in count(num, 45):
    if result%4 == 1:
        break
print(result)
```

3.5 案例精选

- **例3-15** 编写程序，计算组合数 $C(n,i)$ ，即从 n 个元素中任选 i 个，有多少种选法。

根据组合数定义，需要计算3个数的阶乘，在很多编程语言中都很难直接使用整型变量表示大数的阶乘结果，虽然Python并不存在这个问题，但是计算大数的阶乘仍需要相当多的时间。本例提供另一种计算方法：以 $C_{ni}(8,3)$ 为例，按定义式展开如下，对于(5,8]区间的数，分子上出现一次而分母上没出现；(3,5]区间的数在分子、分母上各出现一次；[1,3]区间的数分子上出现一次而分母上出现两次。

$$C_8^3 = \frac{8!}{3! \times (8-3)!} = \frac{8 \times 7 \times 6 \times \textit{5} \times \textit{4} \times \textit{3} \times \textit{2} \times \textit{1}}{3 \times 2 \times 1 \times \textit{5} \times \textit{4} \times \textit{3} \times \textit{2} \times \textit{1}} = \frac{8 \times 7 \times 6}{3 \times 2 \times 1}$$

3.5 案例精选

```
def cni1(n, i):
    if not (isinstance(n,int) and isinstance(i,int) and n>=i):
        print('n and i must be integers and n >= i.')
        return
    result = 1
    Min, Max = sorted((i, n-i))
    for i in range(n, 0, -1):
        if i > Max:
            result *= i
        elif i <= Min:
            result //= i
    return result
```

3.5 案例精选

■ 也可以使用`math`库中的阶乘函数直接按组合数定义实现。

```
>>> def cni2(n, i):  
    import math  
    return int(math.factorial(n)/math.factorial(i)/math.factorial(n-i))  
>>> cni2(6, 2)  
15
```

■ 还可以直接使用Python标准库`itertools`提供的函数。

```
>>> import itertools  
>>> len(tuple(itertools.combinations(range(60), 2)))    # 数字不能特别大  
1770
```

3.5 案例精选

- Python 3.8以及之后的版本中，`math`标准库直接提供了组合数和排列数计算的函数。

```
>>> import math
>>> help(math.comb)
```

```
Help on built-in function comb in module math:
```

```
comb(n, k, /)
```

```
    Number of ways to choose k items from n items without repetition and without order.
```

```
    Evaluates to  $n! / (k! * (n - k)!)$  when  $k \leq n$  and evaluates to zero when  $k > n$ .
```

```
    Also called the binomial coefficient because it is equivalent to the coefficient of k-th term in polynomial expansion of the expression  $(1 + x)^n$ .
```

```
    Raises TypeError if either of the arguments are not integers.
    Raises ValueError if either of the arguments are negative.
```


3.5 案例精选

```
>>> help(math.perm)
```

```
Help on built-in function perm in module math:
```

```
perm(n, k=None, /)
```

```
    Number of ways to choose k items from n items without repetition and with order.
```

```
    Evaluates to  $n! / (n - k)!$  when  $k \leq n$  and evaluates  
    to zero when  $k > n$ .
```

```
    If k is not specified or is None, then k defaults to n  
    and the function returns n!.
```

```
    Raises TypeError if either of the arguments are not integers.  
    Raises ValueError if either of the arguments are negative.
```

3.5 案例精选

- `itertools`模块中的函数`combinations_with_replacement()`可以返回带重复值的组合。

```
from itertools import combinations_with_replacement
```

```
primes = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47)
```

```
# 平方和等于2019的三个素数
```

```
for item in combinations_with_replacement(primes, 3):  
    if sum(map(lambda x:x**2, item)) == 2019:  
        print(item, end=' ')
```

运行结果:

```
(7, 11, 43) (7, 17, 41) (11, 23, 37) (13, 13, 41) (17, 19, 37) (23, 23, 31)
```

3.5 案例精选

- `itertools`还提供了排列函数`permutations()`。

```
>>> import itertools
>>> for item in itertools.permutations(range(1,4), 2):
    print(item)
```

```
(1, 2)
```

```
(1, 3)
```

```
(2, 1)
```

```
(2, 3)
```

```
(3, 1)
```

```
(3, 2)
```

3.5 案例精选

- `itertools`还提供了用于循环遍历可迭代对象元素的函数`cycle()`。

```
>>> import itertools
>>> x = 'Private Key'
>>> y = itertools.cycle(x)                # 循环遍历序列中的元素
>>> for i in range(20):
    print(next(y), end=',')
P,r,i,v,a,t,e, ,K,e,y,P,r,i,v,a,t,e, ,K,
>>> for i in range(5):
    print(next(y), end=',')
e,y,P,r,i,
```

3.5 案例精选

- `itertools`还提供了根据一个序列的值对另一个序列进行过滤的函数`compress()`。

```
>>> x = range(1, 20)
>>> y = (1,0)*9 + (1,)
>>> y
(1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1)
>>> list(itertools.compress(x, y))
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

3.5 案例精选

- `itertools`还提供了根据函数返回值对序列进行分组的函数`groupby()`。

```
>>> def group(v):  
    if v > 10:  
        return 'greater than 10'  
    elif v < 5:  
        return 'less than 5'  
    else:  
        return 'between 5 and 10'
```

```
>>> x = range(20)
```

待分组的数据，要求有序

```
>>> y = itertools.groupby(x, group)
```

根据函数返回值对序列元素进行分组

```
>>> for k, v in y:  
    print(k, ': ', list(v))
```

```
less than 5 : [0, 1, 2, 3, 4]
```

```
between 5 and 10 : [5, 6, 7, 8, 9, 10]
```

```
greater than 10 : [11, 12, 13, 14, 15, 16, 17, 18, 19]
```

3.5 案例精选

- 根据字符种类数量判断密码安全强度。

```
from itertools import groupby
```

```
def rules(ch):
```

```
    if '0'<=ch<='9': return 'digits'
```

```
    if 'a'<=ch<='z': return 'lowercase'
```

```
    if 'A'<=ch<='Z': return 'uppercase'
```

```
    if ch in ',._': return 'punctrations'
```

```
def check(pwd):
```

```
    grade = {1: 'weak', 2: 'below middle', 3: 'above middle', 4: 'strong'}
```

```
    num = len(tuple(groupby(sorted(pwd), key=rules)))
```

```
    return grade.get(num, 'not suitable')
```

```
print(check('abcd'))
```

```
print(check('aB123,'))
```

3.5 案例精选

- **例3-16** 编写程序，计算理财产品收益，假设收益和本金一起滚动。

```
def licai(base, rate, days):  
    # 初始投资金额  
    result = base  
    # 整除，用来计算一年可以滚动多少期  
    times = 365 // days  
    for i in range(times):  
        result = result + result*rate/365*days  
    return result  
  
# 14天理财，利率0.0385，投资10万  
print(licai(100000, 0.0385, 14))
```


3.5 案例精选

- **例3-17** 计算前n个自然数的阶乘之和 $1!+2!+3!+\dots+n!$ 的值。

```
from time import time
from functools import reduce
from operator import mul
from math import factorial
from numpy import cumprod

def func1(n):
    result = 0
    for i in range(1, n+1):
        t = 1
        for j in range(1, i+1):
            t = t * j
        result = result + t
    return result
```

3.5 案例精选

```
def func2(n):
    result = 0
    for i in range(1, n+1):
        result = result+factorial(i)
    return result

def func3(n):                                # 效率最高
    result = 0
    t = 1
    for i in range(1, n+1):
        t = t * i
        result = result + t
    return result

def func4(n):
    return sum(map(factorial, range(1, n+1)))
```

3.5 案例精选

```
def func5(n):  
    return sum(cumprod(range(1,n+1), dtype=object))  
  
def func6(n):  
    return sum(map(lambda x: reduce(mul, range(1,x+1)), range(1, n+1)))  
  
for n in range(3000, 10000, 1000):  
    print(n)  
    values = []  
    for func in (func1, func2, func3, func4, func5, func6):  
        start = time()  
        values.append(func(n))  
        print(time()-start, end='  ')  
    print(len(set(values))==1)
```

3.5 案例精选

3000									
2.207132339477539	0.36505579948425293	0.0030138492584228516	0.3610715866088867	0.008007287979125977	2.190141201019287	True			
4000									
4.876956224441528	0.812824010848999	0.005983591079711914	0.8178110122680664	0.01298832893371582	4.961747646331787	True			
5000									
9.555473804473877	1.5338997840881348	0.00997304916381836	1.5877854824066162	0.021967649459838867	10.041111946105957	True			
6000									
16.663463354110718	2.638932943344116	0.014935493469238281	2.569124221801758	0.03290915489196777	16.797066688537598	True			
7000									
26.068273305892944	3.8976097106933594	0.01798391342163086	3.8836121559143066	0.04290890693664551	26.16902208328247	True			
8000									
38.802325963974	5.632938623428345	0.035936832427978516	5.626992464065552	0.05488944053649902	38.80024242401123	True			
9000									
57.99291110038757	8.344712257385254	0.0359342098236084	8.42447018623352	0.07184934616088867	56.744266986846924	True			

3.5 案例精选

- **例3-18** 验证6174猜想。1955年，卡普耶卡(D.R.Kaprekar)对4位数字进行了研究，发现一个规律：对任意各位数字不相同的4位数，使用各位数字能组成的最大数减去能组成的最小数，对得到的差重复这个操作，最终会得到6174这个数字，并且这个操作最多不会超过7次。

3.5 案例精选

```
from string import digits
from itertools import combinations

for item in combinations(digits, 4):      # 这里用组合和排列是一样的
    times = 0
    while True:
        # 当前选择的4个数字能够组成的最大数和最小数
        big = int(''.join(sorted(item, reverse=True)))
        little = int(''.join(sorted(item)))
        difference = big - little
        times = times + 1
        # 如果最大数和最小数相减得到6174就退出，否则就对得到的差重复这个操作
        # 最多7次，总能得到6174
        if difference == 6174:
            if times > 7:
                print(times)
            break
        else:
            item = str(difference)
```

3.5 案例精选

- **例3-19** 检测序列中的元素是否满足严格升序关系。

```
def lessThan1(seq):
    for index, value in enumerate(seq[:-1]):
        if value >= seq[index+1]:
            return False
    return True

def lessThan2(seq):
    func = lambda x,y: x<y
    return all(map(func, seq[:-1], seq[1:]))

tests = ('abcdeff', [1,2,3,5,4], (3,5,7,9))
for test in tests:
    print(lessThan1(test), lessThan2(test))
```

```
from operator import lt

def lessThan2(seq):
    return all(map(lt, seq[:-1], seq[1:]))
```